Universidad Politécnica de Madrid
ETSI de Telecomunicación

Departamento de Ingeniería Electrónica

Circuitos Electrónicos (CELT)

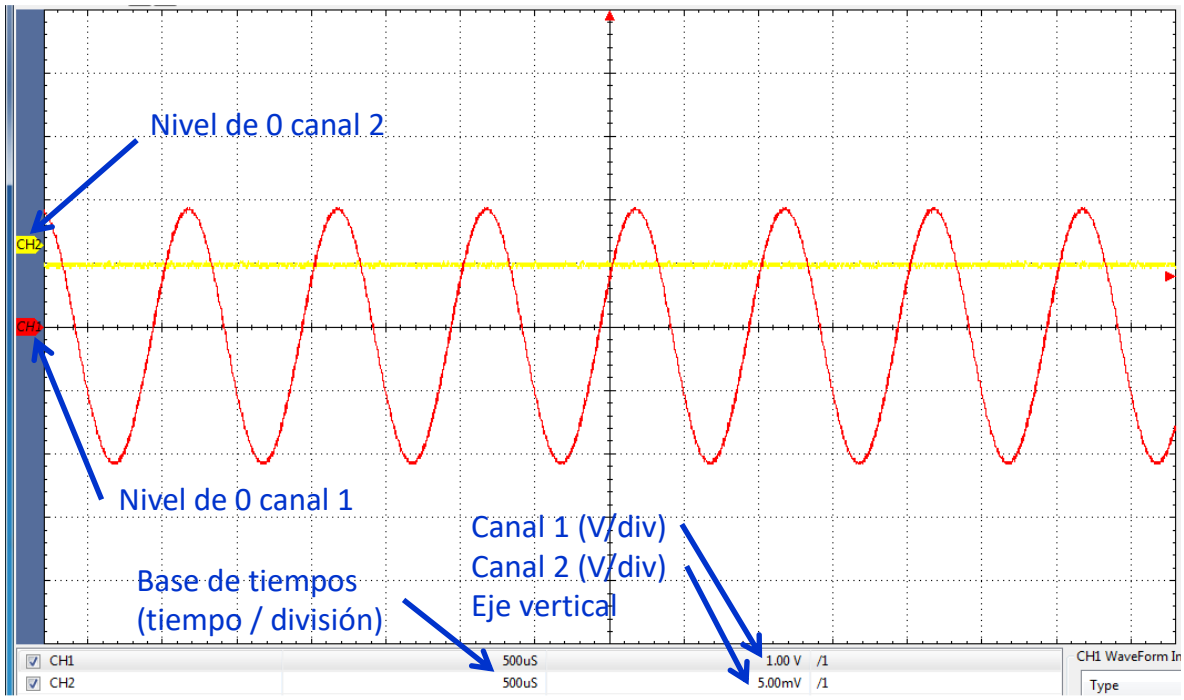
Memoria de la práctica

Curso 2022-2023

Nombre y apellidos alumno/a 1	Adrián Modino Merino
Nombre y apellidos alumno/a 2	Andrés Roques Sanchez
Código de puesto	VT03

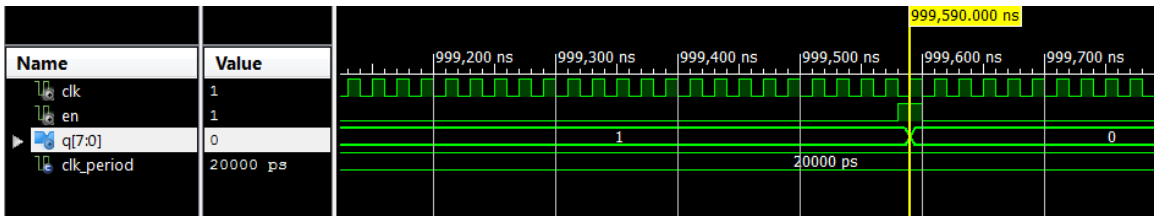
Instrucciones generales

1. Las capturas de pantalla del osciloscopio deben contener la información suficiente para poder deducir la amplitud de las señales y las magnitudes en el eje de tiempos. Para ello debe incluir como mínimo las referencias de los ejes (Voltios por división y tiempo por división, así como el nivel de 0). Vea el siguiente ejemplo:



2. Las expresiones matemáticas y fórmulas pueden entregarse realizadas a mano con letra clara y limpias. Deberá realizar una fotografía bien expuesta y enfocada. También puede emplearse el editor de ecuaciones si se desea.

3. En las capturas de pantalla de las simulaciones VHDL, tenga en cuenta que debido al fondo negro de la pantalla del simulador es difícil que se vean las señales adecuadamente. Se recomienda utilizar la herramienta de recorte de Word y ampliar convenientemente la captura de pantalla para resaltar las regiones importantes. La siguiente figura es un ejemplo de cómo se deben entregar las gráficas de simulación.



Circuito adaptador

1. Detalle los cálculos de la resistencia R1.

$$\frac{Ve - Vi}{\frac{1}{j\omega C}} = \frac{Vi}{R_1}$$

$$j\omega C \cdot R_1 \cdot Ve - j\omega C \cdot R_1 \cdot Vi = Vi$$

$$Ve \cdot j\omega C R_1 = Vi\,(j\omega C R_1 + 1)$$

$$\frac{Vi}{Ve} = \frac{j\omega C R_1}{1 + j\omega C R_1} = 2\pi C R_1 \frac{j \cdot f}{1 + j \cdot 2\pi f C \cdot R_1} =$$

$$2\pi \cdot 10^{-6} \cdot R_1 \frac{j \cdot f}{1 + j \cdot \frac{f}{\frac{1}{2\pi \cdot 10^{-6} \cdot R_1}}} \Big]_{f_p}$$

$$f_p = f_c = 20\ Hz = \frac{1}{2\pi \cdot 10^{-6} \cdot R_1}$$

$$R_1 = \frac{1}{2\pi \cdot 10^{-6} \cdot 20} = \frac{1}{40\pi \cdot 10^{-6}} = 0{,}007957 \cdot 10^{6} = 7{,}96 \cdot 10^{3} \pm 8\ K\Omega$$

(hemos escogido una resistencia de 8,2 KΩ para que la frecuencia de corte no sobrepasara los 20 Hz)

2. Detalle los cálculos de la resistencia R2.

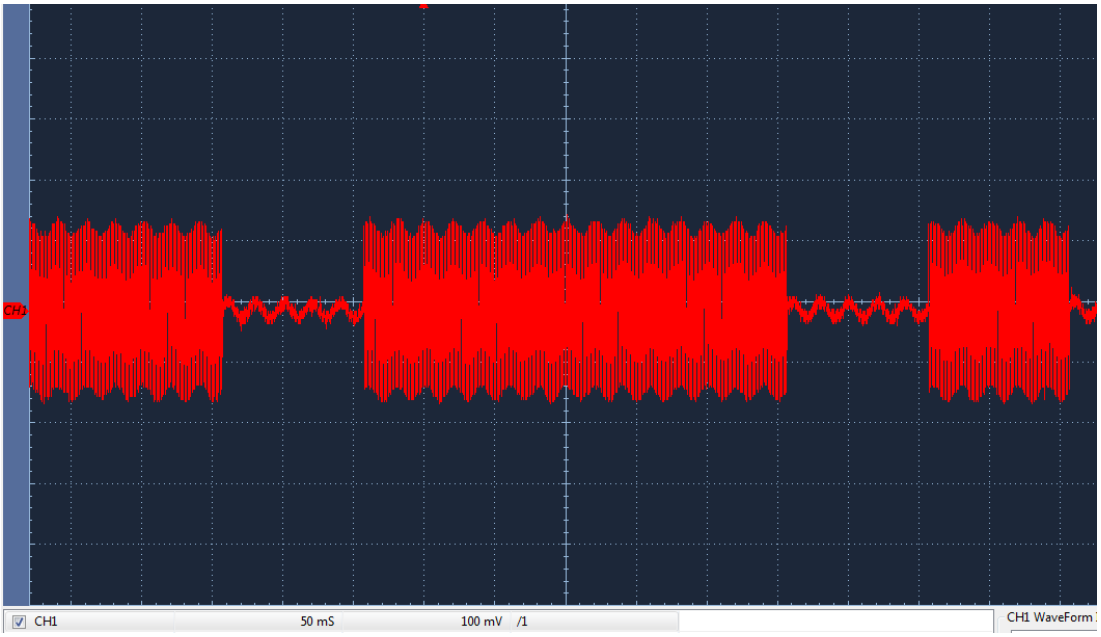
$$G \in [1, 5] = \frac{Vo}{Vi} = 1 + \frac{POT}{R2} = 1 + \frac{10 \cdot 10^{3}}{R2}$$

$$R2 = \frac{POT}{4} = \frac{10 \cdot 10^{3}}{4} = 2{,}5 \cdot 10^{3} = 2{,}5\ K\Omega$$

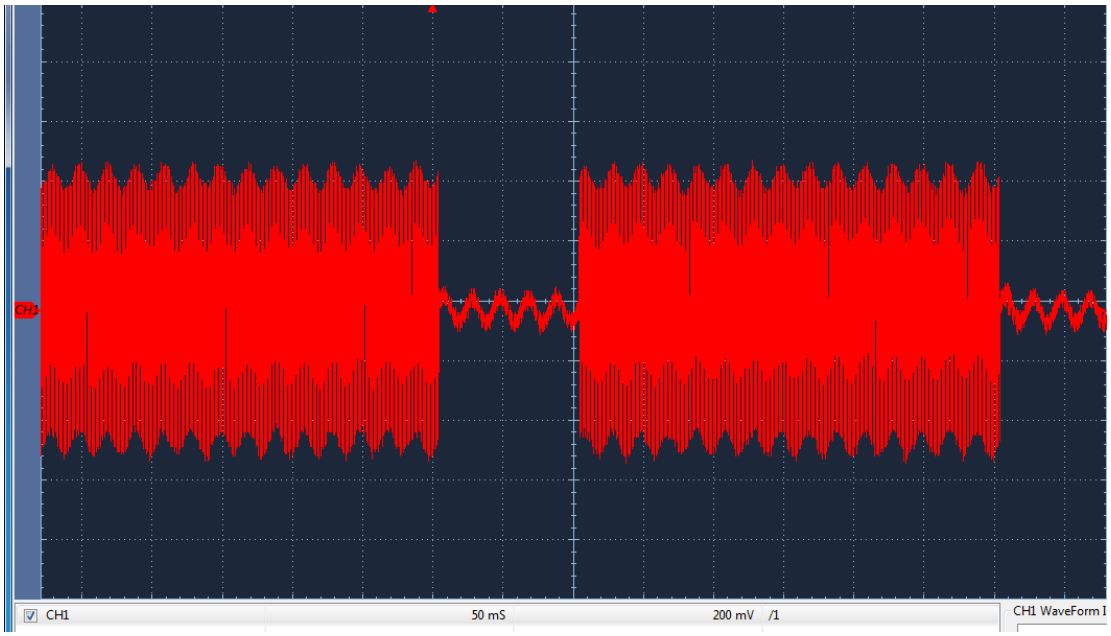
3. Detalle el valor medido para la frecuencia de corte (f_c) del filtro paso alto.

$$f_c = \frac{1}{2\pi \cdot 10^{-6} \cdot R_1} = \frac{1}{2\pi \cdot 10^{-6} \cdot 8{,}2 \cdot 10^{3}} = 19{,}409\ Hz$$

4. Incluya una captura de pantalla del osciloscopio donde se aprecie la señal en el punto Vi del circuito con la amplitud correcta.



5. Incluya una captura de pantalla del osciloscopio donde se aprecie la señal a la salida del circuito (punto Vo) con la amplitud correcta.



Las capturas de pantalla del osciloscopio deben contener la información suficiente para poder deducir la amplitud de las señales y las magnitudes en el eje de tiempos. Para ello debe incluir las referencias de los ejes (Voltios por división y tiempo por división).

Filtro paso banda

1. Detalle el cálculo de la función de transferencia del filtro paso banda.

$$H(jf) = \frac{V_o}{V_i} = \frac{1}{R_1 + \frac{1}{j\omega C_1}} \cdot \frac{R_2 /\!/ \frac{1}{j\omega C_2}}{R_2 + \frac{1}{j\omega C_2}} = \frac{-j\omega C_1}{R_1 j\omega C_1 + 1} \cdot \frac{R_2}{R_2 \cdot j\omega C_2 + 1} = \frac{-j\omega C_1 R_2}{(1 + j\omega C_1 R_1)(1 + j\omega C_2 R_2)} =$$

$$= \overbrace{-2\pi C_1 R_2}^{|K|} \cdot \frac{j \cdot f}{\left(1 + j \cdot \frac{f}{\frac{1}{C_1 R_1 2\pi}}\right)\left(1 + j \cdot \frac{f}{\frac{1}{C_2 R_2 2\pi}}\right)}$$

2. Detalle el cálculo de los valores R1, C1, R2 y C2.

$$|K| = 2\pi C_1 R_2 = 2{,}5 \cdot 10^{-3}$$

$$f_{P_1} = \frac{1}{C_1 R_1 2\pi} = 500$$

$$f_{P_2} = \frac{1}{C_2 R_2 2\pi} = 2000$$

USANDO $R_2 = 1\,k\Omega$

- $C_1 = 0{,}39788\,\mu F \simeq 400\,nF \rightarrow$ en la práctica: 224 nF // 224 nF
- $R_1 = 800{,}0148\,\Omega \simeq 800\,\Omega \rightarrow 820\,\Omega$
- $C_2 = 0{,}0796\,\mu F \simeq 80\,nF \rightarrow$ 47 nF // 47 nF

3. Indique las frecuencias de los polos y ceros de la función de transferencia.

POLOS:

$$f_{P_1} = \frac{1}{C_1 R_1 2\pi} = 500 \quad (\text{dato})$$

$$f_{P_2} = \frac{1}{C_2 R_2 2\pi} = 2000 \quad (\text{dato})$$

CEROS:

$$-2\pi C_1 R_2 \cdot jf = 0 \; ; \; f_{C_1} = 0$$

$$\lim_{f \to \infty} H(jf) = \frac{1}{j \cdot \infty} = 0 \; ; \; f_{C_2} = \infty$$

4. Dibuje los diagramas asintóticos de Bode en módulo y fase del filtro empleando las plantillas de las páginas siguientes. Indique las frecuencias de los polos, las pendientes de las atenuaciones, el máximo valor de la ganancia y las pendientes de la fase. HECHO

5. Rellene la tabla para la caracterización del filtro de la página siguiente.

   HECHO

6. Superponga los puntos de las medidas sobre el diagrama asintótico de Bode de modo que se vean las gráficas de ganancia y fase. HECHO

7. Indique el valor de la frecuencia central (f_0) medida con precisión.

   La frecuencia central (1000 Hz) se da a ganancia máxima que en nuestro caso es 0,94 (-0,537 dB) debido a que los componentes serían:

   $C_1 \simeq 400\ nF$

   $R_1 \simeq 800\ \Omega$

   $C_2 \simeq 80\ nF$

   Y lo más cercano a esos valores que hemos encontrado es:

   $C_1 = 224\ nF + 224\ nF = 448\ nF$

   $R_1 = 820\ \Omega$

   $C_2 = 47\ nF + 47\ nF = 94\ nF$

8. Incluya una captura de pantalla del osciloscopio donde se aprecie la señal a la salida del filtro paso banda (punto v_o) cuando se encuentra conectado con la etapa anterior y todo ello alimentado con la señal de audio del ordenador.

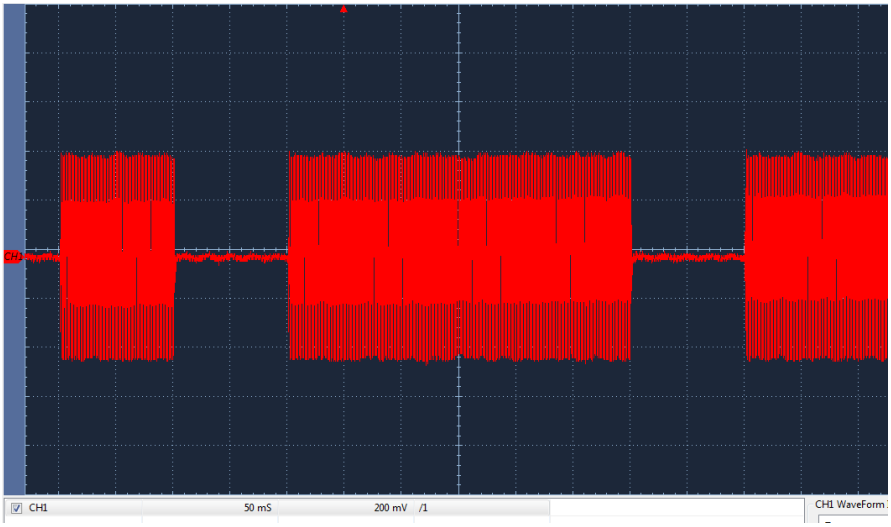
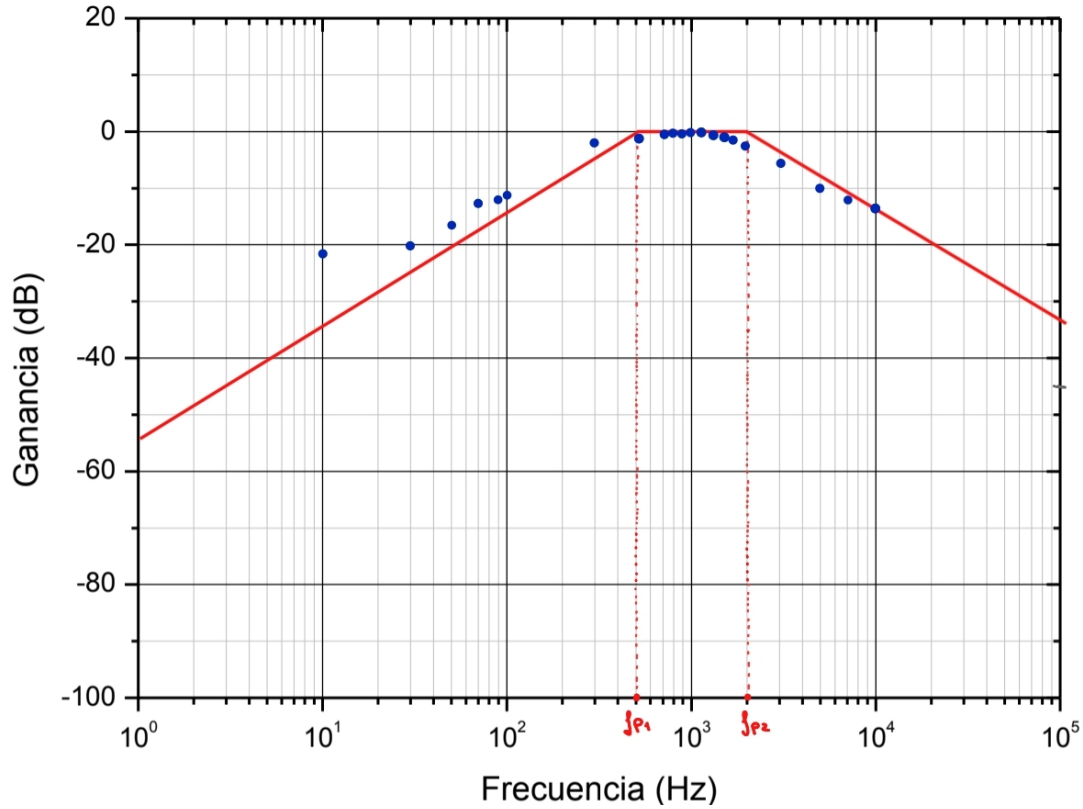


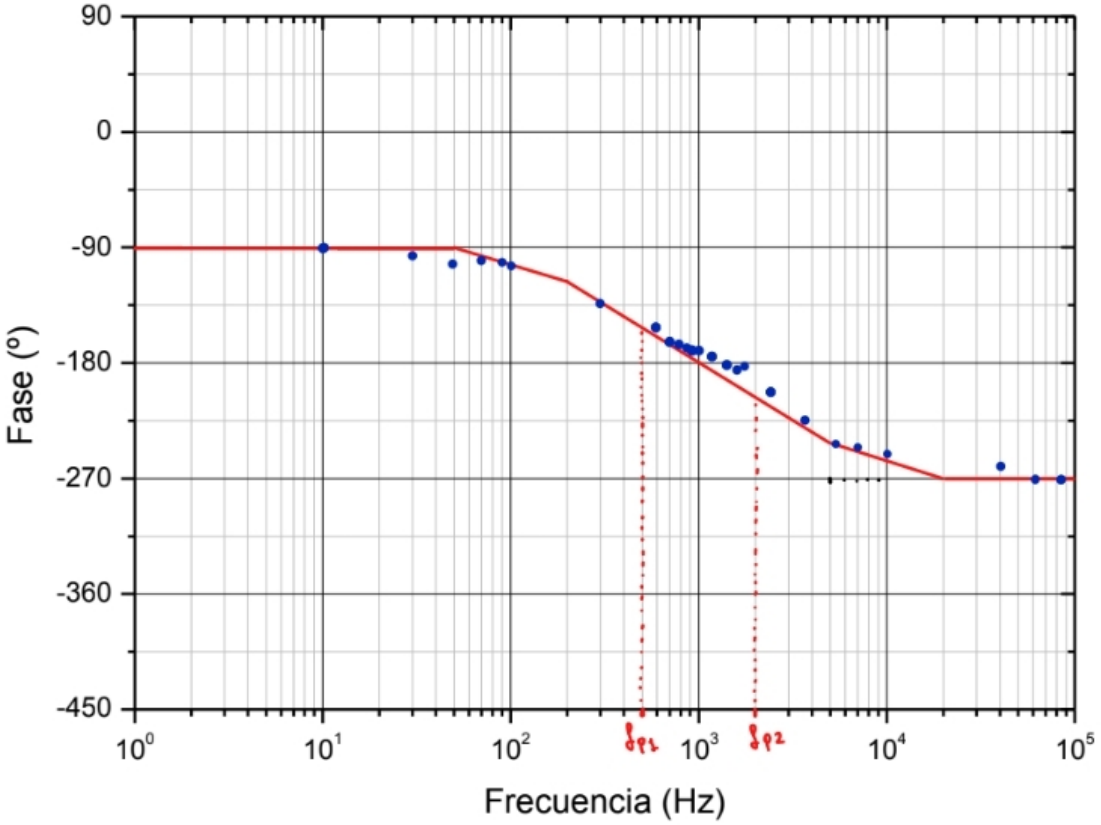
Tabla para la medida del filtro paso banda

Frecuencia (Hz)	Periodo T (s)	Entrada (Vpp)	Salida (Vpp)	Ganancia	Ganancia (dB)	ΔT (s)	Fase (º)
10	0,1000000	1	0,07	0,07	-23,0980392	0,025	-90,00
30	0,0333333	1	0,085	0,085	-21,41162149	0,0093333	-100,80
50	0,0200000	1	0,13	0,13	-17,72113295	0,007	-126,00
70	0,0142857	1	0,185	0,185	-14,65656543	0,0038857	-97,92
90	0,0111111	1	0,24	0,24	-12,39577517	0,0032111	-104,04
100	0,0100000	1	0,26	0,26	-11,70053304	0,003	-108,00
300	0,0033333	1	0,62	0,62	-4,15216621	0,0012333	-133,20
500	0,0020000	1	0,81	0,81	-1,830299622	0,00084	-151,20
700	0,0014286	1	0,9	0,9	-0,915149811	0,0006786	-171,00
800	0,0012500	1	0,92	0,92	-0,724243453	0,00060	-172,80
900	0,0011111	1	0,93	0,93	-0,630341029	0,0005511	-178,56
1.000	0,0010000	1	0,94	0,94	-0,537442928	0,0005	-180,00
1.100	0,0009091	1	0,92	0,92	-0,724243453	0,0004591	-181,80
1.300	0,0007692	1	0,9	0,9	-0,915149811	0,0004192	-196,20
1.500	0,0006667	1	0,86	0,86	-1,310030975	0,0003767	-203,40
1.700	0,0005882	1	0,81	0,81	-1,830299622	0,0003382	-207,00
2.000	0,0005000	1	0,78	0,78	-2,158107946	0,00028	-201,60
3.000	0,0003333	1	0,61	0,61	-4,2934033	0,0002083	-225,00
5.000	0,0002000	1	0,42	0,42	-7,535014192	0,0001340	-241,20
7.000	0,0001429	1	0,31	0,31	-10,17276612	0,0000979	-246,60
10.000	0,0001000	1	0,23	0,23	-12,76544328	0,00007	-252,00
30.000	0,0000333	1	0,08	0,08	-21,93820026	0,0000243	-262,80
50.000	0,0000200	1	0,055	0,055	-25,19274621	0,0000150	-270,00
70.000	0,0000143	1	0,042	0,042	-27,53501419	0,0000107	-269,28

Frecuencia central (f_0) medida	**1000**

DIAGRAMAS ASINTÓTICOS DE BODE Y MEDIDAS, FILTRO PASO BANDA





Amplificador y rectificador

1. Razone el valor de la ganancia del amplificador (G).

$$G = \frac{V_s}{V_i} = \frac{5}{1} = 5$$

2. Detalle la deducción de la expresión de la ganancia G.

$$\frac{0 - V_i}{R_1} = \frac{V_i - V_s}{R_2}$$

$$-R_2 \cdot V_i = R_1 \cdot V_i - R_1 \cdot V_s$$

$$V_i (R_2 + R_1) = R_1 \cdot V_s$$

$$G = \frac{V_s}{V_i} = \frac{R_2 + R_1}{R_1}$$

3. Detalle el cálculo de las resistencias R1 y R2.

$$\frac{0 - V_i}{R_1} = \frac{V_i - V_s}{R_2}$$

$$\frac{-1}{R_1} = \frac{1 - 5}{R_2}$$

$$R_2 = 4 \cdot R_1$$

En la práctica:

$R_1 = 13\ K\Omega$

$R_2 = 51{,}3\ K\Omega$

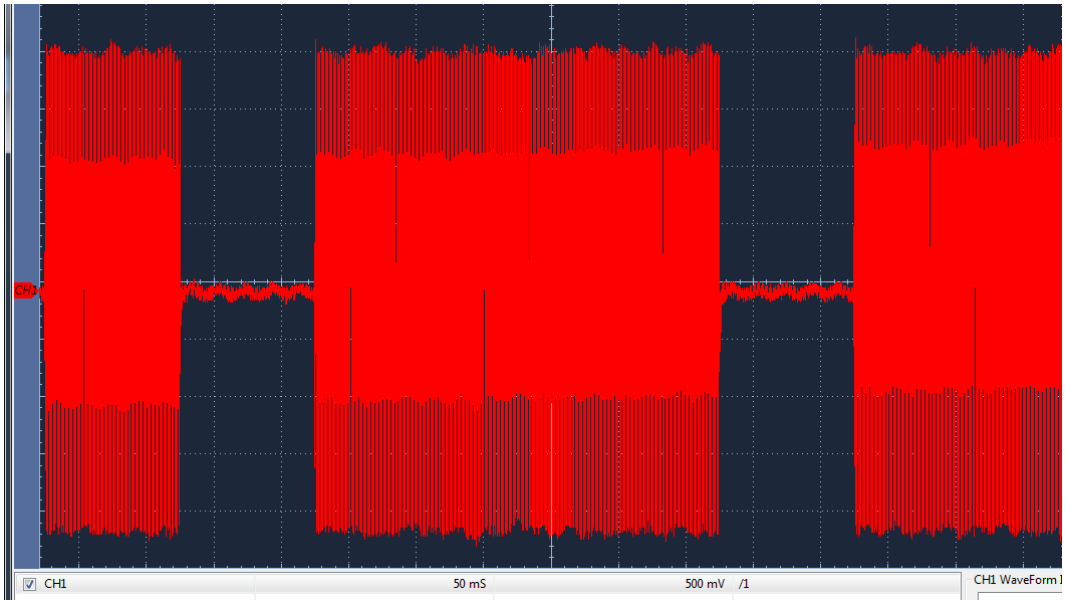
4. Detalle el cálculo de la resistencia R.

$$I_D = \frac{V_o}{R} \longrightarrow R = \frac{V_o}{I_D} \begin{cases} I_D = 1mA : \frac{1{,}8\ V}{1mA} = 1{,}8\ K\Omega \\ I_D = 5mA : \frac{1{,}8\ V}{5mA} = 360\ \Omega \end{cases} \quad R \in (360\ \Omega , 1800\ \Omega)$$

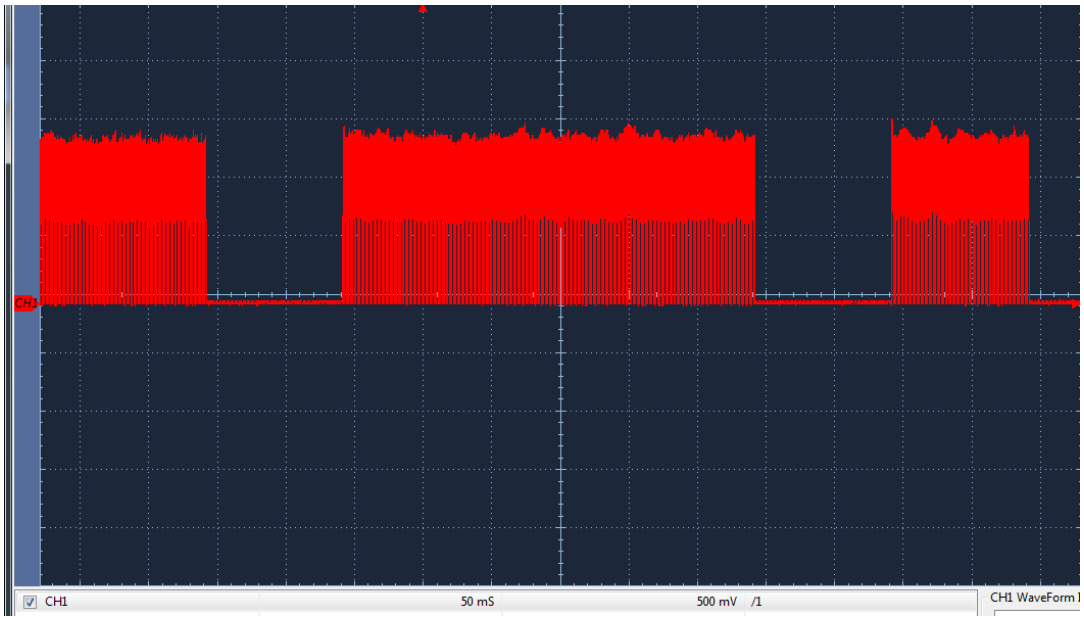
En la práctica:

$R = 1\ K\Omega$

5. Incluya una captura de pantalla del osciloscopio donde se aprecie la señal a la salida del amplificador (punto v_S del circuito) cuando se encuentra conectado con las etapas anteriores y todo ello alimentado con la señal Morse del audio del ordenador.



6. Incluya una captura de pantalla del osciloscopio donde se aprecie la señal a la salida del rectificador (punto v_o del circuito) cuando el circuito se encuentra conectado con las etapas anteriores y todo ello alimentado con la señal Morse del audio del ordenador.



Las capturas de pantalla del osciloscopio deben contener la información suficiente para poder deducir la amplitud de las señales y las magnitudes en el eje de tiempos. Para ello debe incluir las referencias de los ejes (Voltios por división y tiempo por división).

Filtro paso bajo

1. Detalle el cálculo de los valores *m* y *n*.

$$Q = \frac{\sqrt{m \cdot n}}{m + 1} = \frac{1}{\sqrt{2}} \quad \begin{cases} m = 1 \\ n = 2 \end{cases}$$

2. Detalle el cálculo de los valores de R y C.

$$f_c = 100\ Hz = \frac{1}{2\pi\sqrt{m \cdot n} \cdot RC} \longrightarrow RC = 0{,}0011254 \quad \begin{cases} C = 20\ nF \\ R = 56\ K\Omega \end{cases}$$

En la práctica:

$R = 51{,}3\ K\Omega$

$m \cdot R = 51\ K\Omega$

$C = 22\ nF$

$n \cdot C = 47\ nF$

3. Rellene la tabla para la caracterización del filtro en la página siguiente.

   HECHO

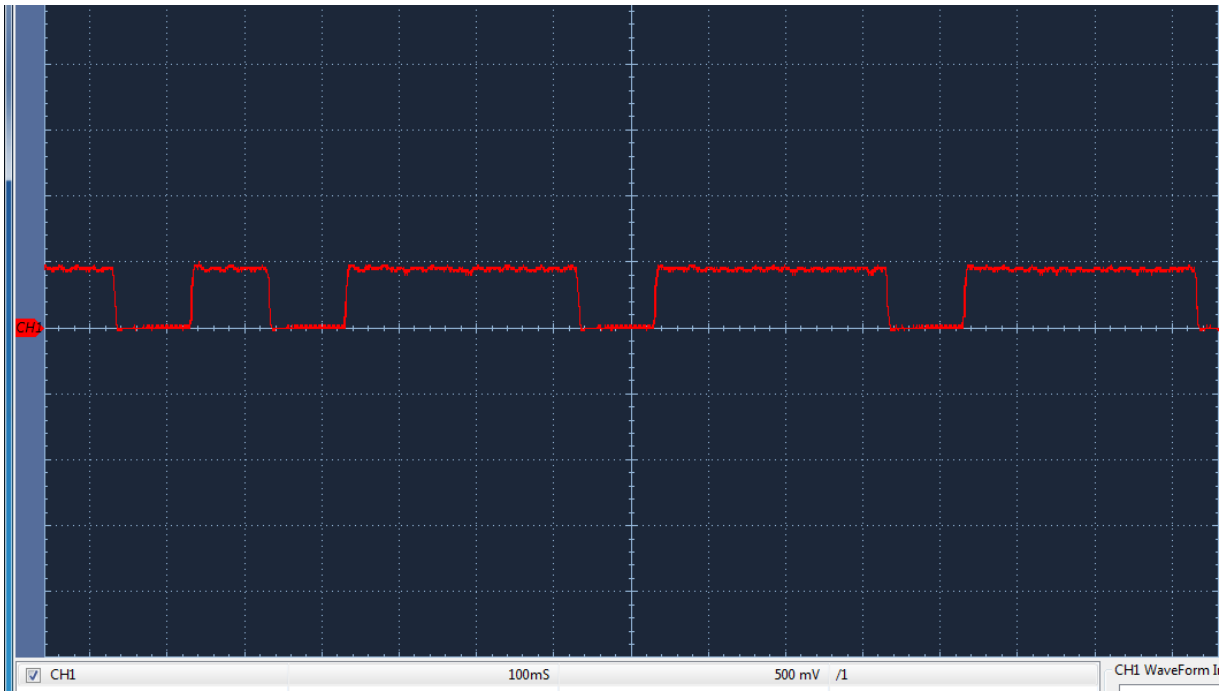
4. Dibuje los puntos de las medidas sobre las gráficas de ganancia y fase que se proporcionan en las páginas siguientes.

   HECHO

5. Indique el valor de la frecuencia de corte (f_c) medida con precisión.

$$f_c = \frac{1}{2\pi\sqrt{m \cdot n} \cdot RC} \quad \begin{cases} m = 1 \\ n = 2 \\ R = 51{,}3\ K\Omega \\ C = 22\ nF \end{cases} = 99{,}7106\ Hz$$

6. Incluya una captura de pantalla del osciloscopio donde se aprecie la señal a la salida del filtro paso bajo cuando se encuentra conectado con las etapas anteriores y todo ello alimentado con la señal Morse del audio del ordenador.



Las capturas de pantalla del osciloscopio deben contener la información suficiente para poder deducir la amplitud de las señales y las magnitudes en el eje de tiempos. Para ello debe incluir las referencias de los ejes (Voltios por división y tiempo por división).

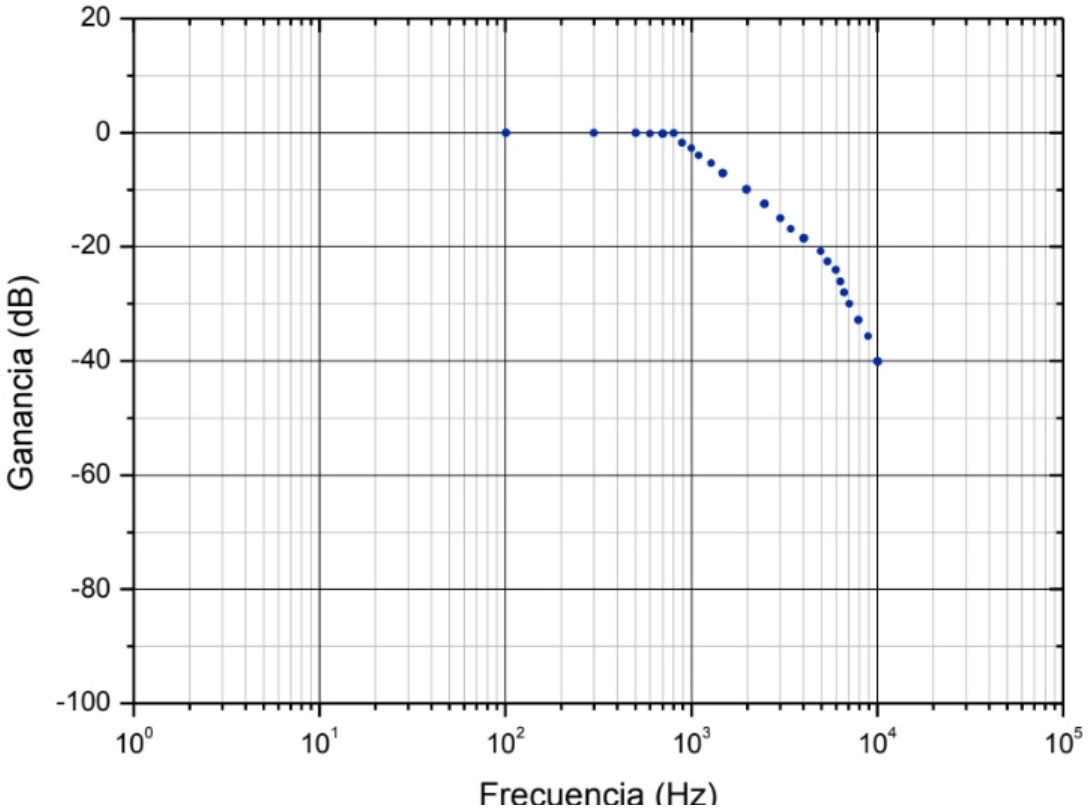
Tabla para la medida del filtro paso bajo

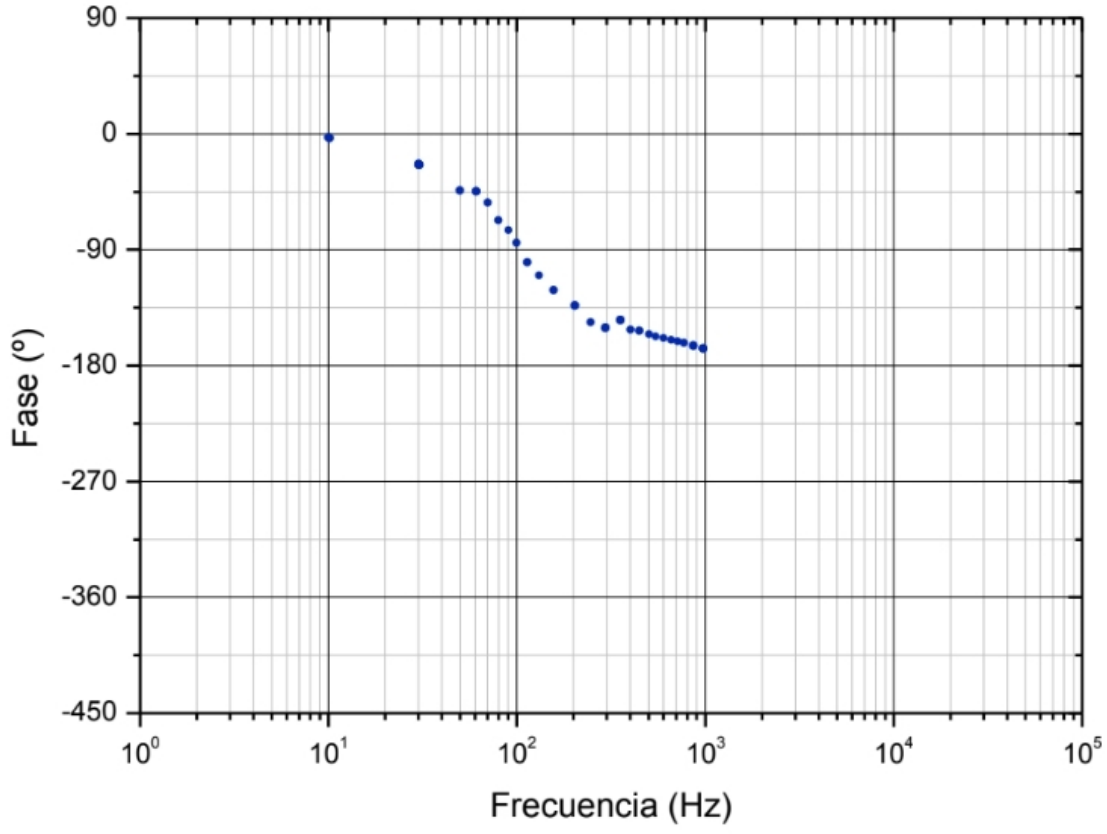
Frecuencia (Hz)	Periodo T (s)	Entrada (Vpp)	Salida (Vpp)	Ganancia	Ganancia (dB)	ΔT (s)	Fase (º)
10	0,1	1	1	1	0	0,002	-7,2
30	0,033333333	1	0,992	0,992	-0,069766557	0,0024	-25,92
50	0,02	1	0,985	0,985	-0,13127539	0,003	-54
60	0,016666667	5	4,8	0,96	-0,354575339	0,0026	-56,16
70	0,014285714	1	0,92	0,92	-0,724243453	0,0024	-60,48
80	0,0125	5	4,2	0,96	-0,354575339	0,0026	-74,88
90	0,011111111	1	0,8	0,8	-1,93820026	0,0025	-81
100	0,01	1	0,71	0,71	-2,974833026	0,0025	-90
110	0,009090909	5	3,12	0,624	-4,096308206	0,0025	-99
130	0,007692308	5	2,48	0,496	-6,09036647	0,0024	-112,32
150	0,006666667	5	2	0,4	-7,958800173	0,00225	-121,5
200	0,005	5	1,2	0,24	-12,39577517	0,00185	-133,2
250	0,004	5	0,8	0,16	-15,91760035	0,00165	-148,5
300	0,003333333	5	0,552	0,1104	-19,14061853	0,001425	-153,9
350	0,002857143	5	0,418	0,0836	-21,55587445	0,0012	-151,2
400	0,0025	5	0,32	0,064	-23,87640052	0,0011	-158,4
450	0,002222222	5	0,256	0,0512	-25,81460078	0,001	-162
500	0,002	15	0,6	0,04	-27,95880017	0,0009	-162
550	0,001818182	15	0,496	0,033066667	-29,61219165	0,00082	-162,36
600	0,001666667	15	0,424	0,028266667	-30,97450805	0,00076	-164,16
650	0,001538462	15	0,36	0,024	-32,39577517	0,000715	-167,31
700	0,001428571	15	0,31	0,020666667	-33,6945913	0,00067	-168,84
900	0,001111111	15	0,19	0,012666667	-37,94675316	0,000525	-170,1
1000	0,001	15	0,154	0,010266667	-39,77141076	0,00048	-172,8

Frecuencia de corte (f_c) medida	**101**

Esta diferencia con la calculada teóricamente puede ser provocada por fallos de precisión en la medición

MEDIDA DEL FILTRO PASO BAJO



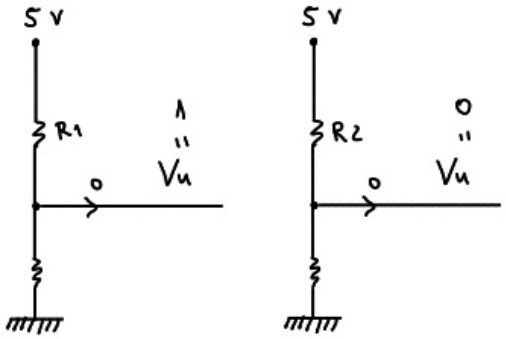


Comparador

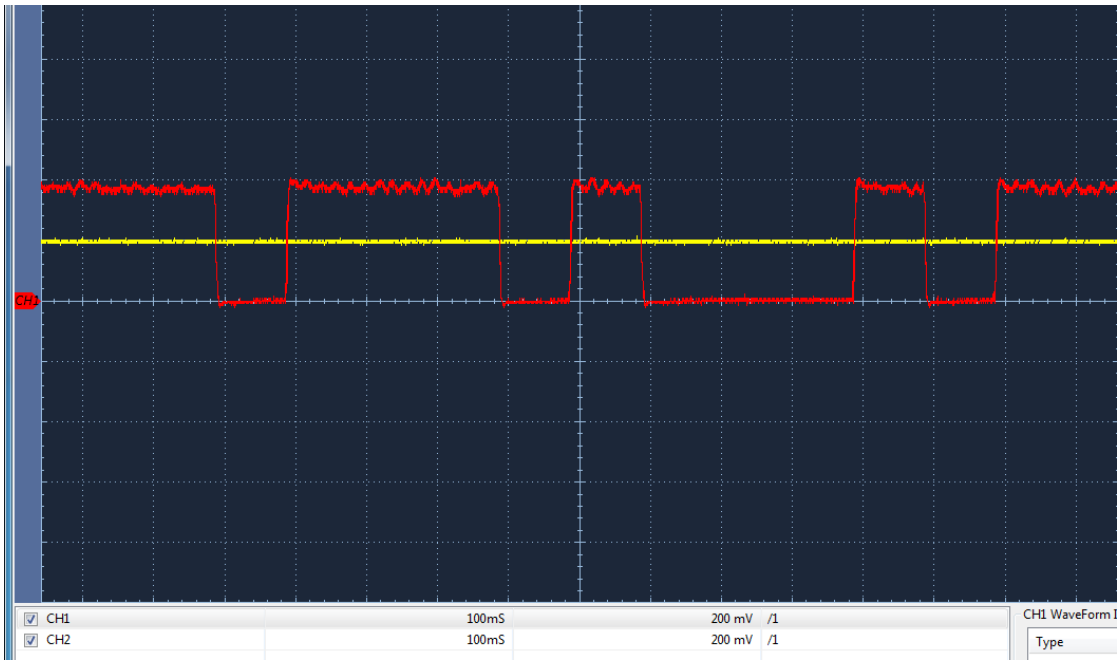
1. Detalle el cálculo de la resistencia R1.

$$\frac{5 - V_u}{R_1} = \frac{V_u}{10\,K}$$

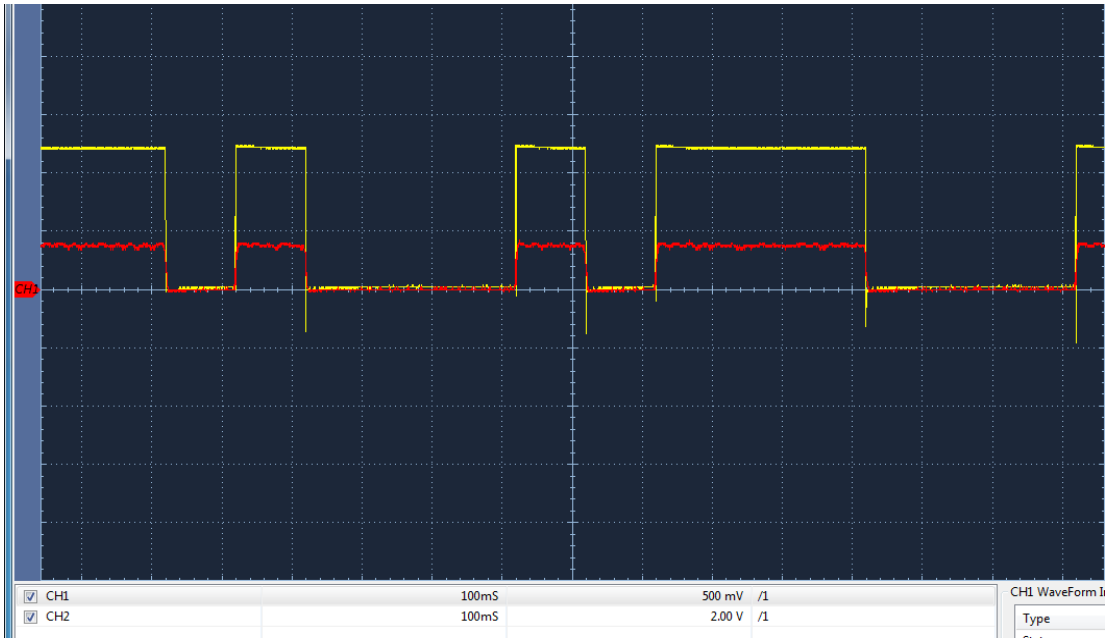
$$R_1 = 40\ k\Omega$$



2. Incluya la captura de pantalla del osciloscopio donde puedan observarse simultáneamente la señal procedente del filtro paso bajo y la tensión del umbral correctamente ajustado. Ambos canales deben tener la misma escala y el mismo nivel de 0.



3. Incluya la captura de pantalla del osciloscopio donde puedan observarse simultáneamente la señal procedente del filtro junto con la señal a la salida del comparador.

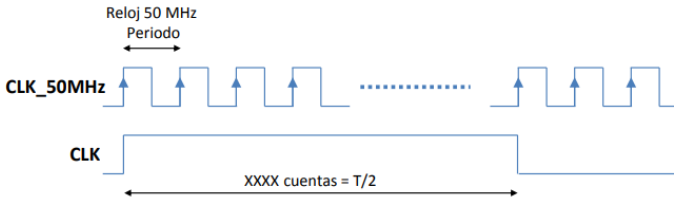


Las capturas de pantalla del osciloscopio deben contener la información suficiente para poder deducir la amplitud de las señales y las magnitudes en el eje de tiempos. Para ello debe incluir las referencias de los ejes (Voltios por división y tiempo por división).

Divisor del reloj

1. Detalle el cálculo de la constante XXXX

XXXX = 25000 ya que, la señal CLK que queremos conseguir CLK es de 1kHz, es decir, de periodo 1ms, y la señal que tenemos CLK_50MHz, un periodo de 20 ns. Asi, al igual que expresa el dibujo, XXXX:



$XXXX*20*10^{-9}=0.5*1*10^{-3}$ → $XXXX=(10^{-3}*0.5)/(20*10^{-9})$ → XXXX= 25000

2. Detalle el código VHDL debidamente comentado.

```
entity div_reloj is
    Port ( CLK_50MHz : in  STD_LOGIC;       -- Entrada reloj de la FPGA 50 MHz
           CLK       : out  STD_LOGIC);     -- Salida reloj a 1 KHz
end div_reloj;

architecture a_div_reloj of div_reloj is

signal contador : STD_LOGIC_VECTOR (15 downto 0);      -- contador
signal frec_div : STD_LOGIC;                    -- señal de frecuencia dividida

begin

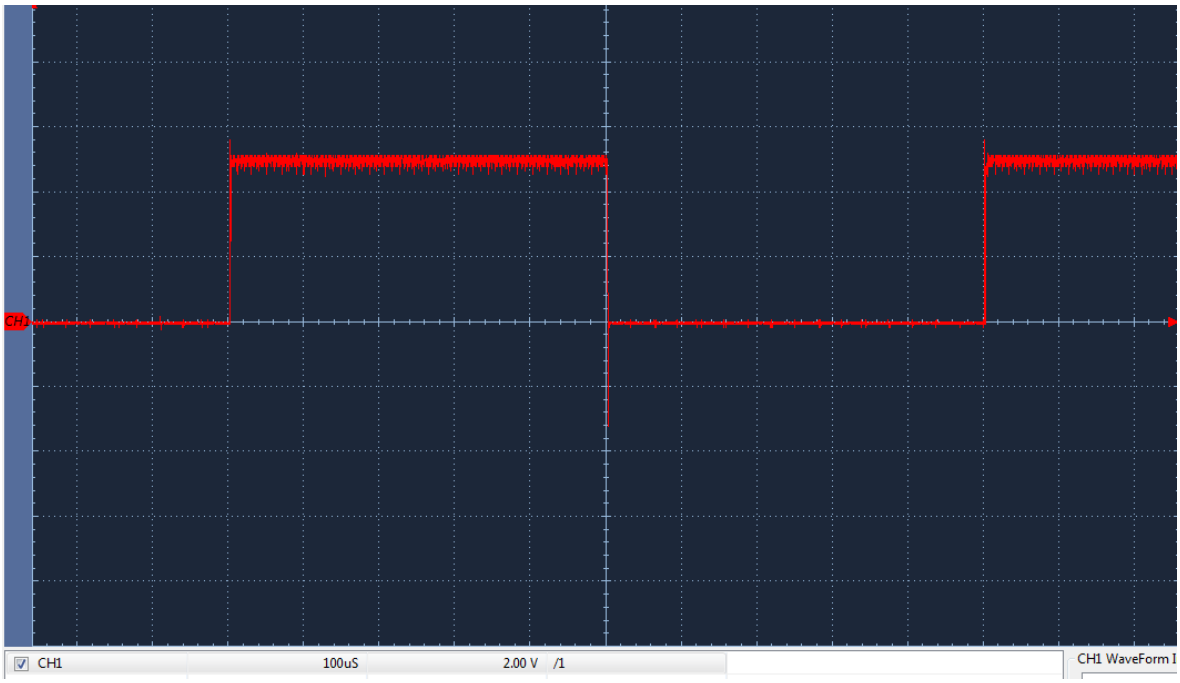
process(CLK_50MHz)
  begin
  if (CLK_50Mhz'event and CLK_50MHz='1') then  -- En cada flanco de subida
    contador<=contador+1;                      -- Incrementar el contador
    if (contador>=25000) then                 -- cuando alcanza el valor 2500
      contador<=(others=>'0');                 -- Poner el contador a 0 y…
          frec_div<=not frec_div;          -- intercambiar el valor de frec_div
    end if;
  end if;

  end process;

CLK<=frec_div;              -- La salida que nos interesa es la señal frec_div

end a_div_reloj;
```

3. Incluya la captura de pantalla del osciloscopio donde pueda observarse la señal de reloj de 1 kHz con niveles de 0 y 5V.



Módulo de visualización

DATOS PARA LA MEMORIA

1. Detalle el código VHDL de los cinco módulos debidamente comentado:

decodmorsea7s.vhd, MUX4x8.vhd, refresco.vhd, rdesp_disp.vhd, visualizacion.vhd

decodmorsea7s.vhd

```
entity decodmorsea7s is
    Port ( CODIGO     : in   STD_LOGIC_VECTOR (7 downto 0);     -- Entrada del
código morse
           SEGMENTOS : out  STD_LOGIC_VECTOR (0 to 6));  -- Salidas al display
end decodmorsea7s;

architecture a_decodmorsea7s of decodmorsea7s is

begin
-- escribimos como debe salir cada letra en el display poniendo a 0 o a 1 los
segmentos correspondientes
   with CODIGO select SEGMENTOS<=                    -- abcdefg
    "0001000"  when "01001000", -- A (2 .-   )
    "1100000"  when "10010000", -- B (4 -... )
    "0110001"  when "10010100", -- C (4 _._. )
    "1000010"  when "01110000", -- D (3 _..  )          a
    "0110000"  when "00100000", -- E (1 .    )        -----
    "0111000"  when "10000100", -- F (4 ..-. )       |     |
    "0000100"  when "01111000", -- G (3 --.  )     f |     | b
    "1001000"  when "10000000", -- H (4 .... )       |  g  |
    "1101111"  when "01000000", -- I (2 ..   )        -----
    "1000011"  when "10001110", -- J (4 .--- )       |     |
    "1111111"  when "01110100", -- K (3 -.-  )     e |     | c
    "1110001"  when "10001000", -- L (4 .-.. )       |     |
    "1111111"  when "01011000", -- M (2 --   )        -----
    "1101010"  when "01010000", -- N (2 -.   )          d
    "1100010"  when "01111100", -- O (3 ---  )
    "0011000"  when "10001100", -- P (4 .--. )   los segmentos
    "0001100"  when "10011010", -- Q (4 --.- )   se encienden
    "1111010"  when "01101000", -- R (3 .-.  )   con un '0'
    "0100100"  when "01100000", -- S (3 ...  )
    "1110000"  when "00110000", -- T (1 -    )
    "1100011"  when "01100100", -- U (3 ..-  )
    "1111111"  when "10000010", -- V (4 ...- )
    "1111111"  when "01101100", -- W (3 .--  )
    "1111111"  when "10010010", -- X (4 -..- )
    "1000100"  when "10010110", -- Y (4 -.-- )
    "1111111"  when "10011000", -- Z (4 --.. )
    "1001111"  when "10101111", -- 1 (5 .----)
```

```
    "0010010"  when "10100111", -- 2 (5 ..---)
    "0000110"  when "10100011", -- 3 (5 ...--)
    "1001100"  when "10100001", -- 4 (5 ....-)
    "0100100"  when "10100000", -- 5 (5 .....)
    "0100000"  when "10110000", -- 6 (5 -....)
    "0001111"  when "10111000", -- 7 (5 --...)
    "0000000"  when "10111100", -- 8 (5 ---..)
    "0001100"  when "10111110", -- 9 (5 ----.)
    "0000001"  when "10111111", -- 0 (5 -----)
    "1111111" when others;

end a_decodmorsea7s;
```

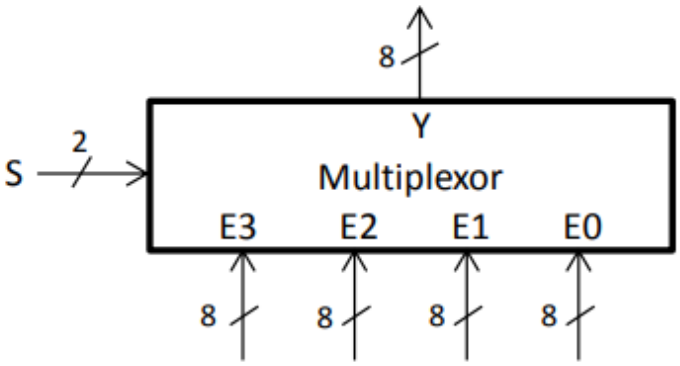
MUX4x8.vhd

```
entity MUX4x8 is
    Port ( E0 : in  STD_LOGIC_VECTOR (7 downto 0); -- Entrada 0
           E1 : in  STD_LOGIC_VECTOR (7 downto 0); -- Entrada 1
           E2 : in  STD_LOGIC_VECTOR (7 downto 0); -- Entrada 2
           E3 : in  STD_LOGIC_VECTOR (7 downto 0); -- Entrada 3
           S : in  STD_LOGIC_VECTOR (1 downto 0);  -- Señal de control
           Y : out  STD_LOGIC_VECTOR (7 downto 0)); -- Salida
end MUX4x8;

architecture a_MUX4x8 of MUX4x8 is

begin
      -- La selección de entrada a poner en la salida..
    Y <= E0 when S = "00" else
         E1 when S = "01" else
         E2 when S = "10" else
         E3 when S = "11" else
         "11111111";

end a_MUX4x8;
```



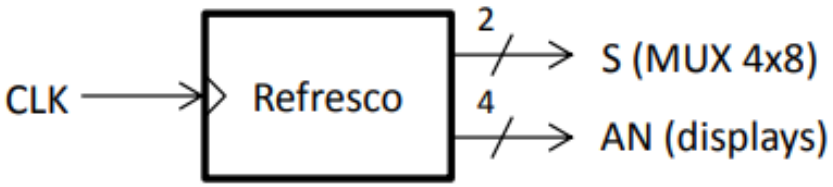
refresco.vhd

```
entity refresco is
    Port ( CLK : in  STD_LOGIC;                          -- reloj de refresco
           S : out  STD_LOGIC_VECTOR (1 downto 0);    -- Control para el mux
           AN  :  out    STD_LOGIC_VECTOR  (3  downto  0));  --  Control  displays
individuales
end refresco;

architecture a_refresco of refresco is
   signal sel_mux : unsigned (1 downto 0) :="00"; -- señal para controlar el
   mux
   begin
         process (CLK) -- el proceso se dispara cada vez que cambia el reloj
         begin
               if CLK'event and CLK='1' then -- En cada flanco de subida
                     sel_mux<=sel_mux+1;
               end if;
         end process;

   S <= STD_LOGIC_VECTOR(sel_mux); -- Se cablea S con la señal sel_mux

   AN<="0111" when sel_mux = "00" else -- La señal AN se cablea de modo que se
activa
          "1011" when sel_mux = "01" else -- cada display
          "1101" when sel_mux = "10" else -- en función del valor de sel_mux
          "1110" when sel_mux = "11"; -- Los displays se encienden con un '0'
end a_refresco;
```



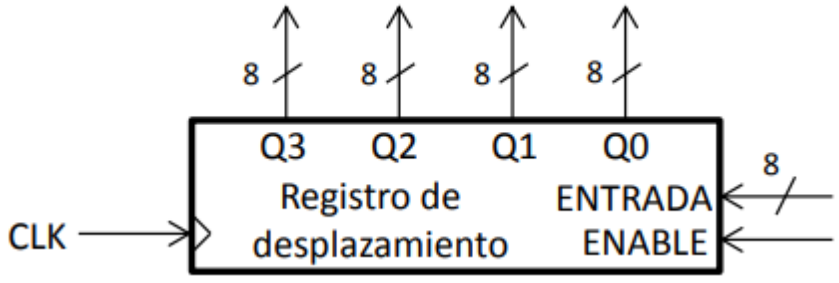
rdesp_disp.vhd

```
entity rdesp_disp is
    Port ( CLK     : in   STD_LOGIC;                       -- entrada de reloj
           ENABLE  : in   STD_LOGIC;                       -- enable
           ENTRADA : in   STD_LOGIC_VECTOR (7 downto 0);   -- entrada de datos
           Q0      : out  STD_LOGIC_VECTOR (7 downto 0);   -- salida Q0
           Q1      : out  STD_LOGIC_VECTOR (7 downto 0);   -- salida Q1
           Q2      : out  STD_LOGIC_VECTOR (7 downto 0);   -- salida Q2
           Q3      : out  STD_LOGIC_VECTOR (7 downto 0));  -- salida Q3
end rdesp_disp;

architecture a_rdesp_disp of rdesp_disp is
   signal A0 : STD_LOGIC_VECTOR (7 downto 0) := (others=>'0');
   signal A1 : STD_LOGIC_VECTOR (7 downto 0) := (others=>'0');
   signal A2 : STD_LOGIC_VECTOR (7 downto 0) := (others=>'0');
   signal A3 : STD_LOGIC_VECTOR (7 downto 0) := (others=>'0');

begin

   process (CLK) -- el proceso se dispara cada vez que cambia el reloj
   begin
         if CLK'event and CLK='1' then
               if ENABLE ='1' then  -- proceso síncrono, por eso el enable se
               evalua tras el flanco de subida del reloj
                     A3<=A2; --las señales toman el valor de la anterior para
producir el efecto de desplazamiento hacia la derecha
                     A2<=A1;
                     A1<=A0;
                     A0<=ENTRADA;
               end if;
         end if;
   end process;
         Q0 <= A0;  -- cableamos las salidas con las señales Ax
         Q1 <= A1;
         Q2 <= A2;
         Q3 <= A3;
end a_rdesp_disp;
```



visualizacion.vhd

```
entity visualizacion is
  Port ( ENTRADA : in STD_LOGIC_VECTOR (7 downto 0);    -- Entrada sig.
         ENABLE  : in STD_LOGIC;           -- Activación para desplazamiento
         CLK     : in STD_LOGIC;                        -- Entrada de reloj
         SEG7    : out STD_LOGIC_VECTOR (0 to 6);       -- Segmentos displays
         AN     : out STD_LOGIC_VECTOR (3 downto 0));  -- Activaci□n displays
end visualizacion;

architecture a_visualizacion of visualizacion is

-- COMPONENTES

component decodmorsea7s is
    Port ( CODIGO    : in  STD_LOGIC_VECTOR (7 downto 0); -- Entrada del morse
           SEGMENTOS : out  STD_LOGIC_VECTOR (0 to 6));  -- Salidas al display
end component;

component refresco is
    Port ( CLK : in  STD_LOGIC;
           S : out  STD_LOGIC_VECTOR (1 downto 0);
           AN : out  STD_LOGIC_VECTOR (3 downto 0));
end component;

component MUX4x8 is
   Port ( E0 : in  STD_LOGIC_VECTOR (7 downto 0); -- Entrada 0
          E1 : in  STD_LOGIC_VECTOR (7 downto 0); -- Entrada 1
          E2 : in  STD_LOGIC_VECTOR (7 downto 0); -- Entrada 2
          E3 : in  STD_LOGIC_VECTOR (7 downto 0); -- Entrada 3
          S : in  STD_LOGIC_VECTOR (1 downto 0);  -- Señal de control
          Y : out  STD_LOGIC_VECTOR (7 downto 0)); -- Salida
end component;

component rdesp_disp is
   Port ( CLK     : in   STD_LOGIC;                      -- entrada de reloj
          ENABLE  : in   STD_LOGIC;                       -- enable
          ENTRADA : in   STD_LOGIC_VECTOR (7 downto 0);   -- entrada de datos
          Q0      : out  STD_LOGIC_VECTOR (7 downto 0);   -- salida Q0
          Q1      : out  STD_LOGIC_VECTOR (7 downto 0);   -- salida Q1
          Q2      : out  STD_LOGIC_VECTOR (7 downto 0);   -- salida Q2
          Q3      : out  STD_LOGIC_VECTOR (7 downto 0));  -- salida Q3
end component;

signal ADRI1 : STD_LOGIC_VECTOR (7 downto 0);   --señales de intercableado
signal ADRI2 : STD_LOGIC_VECTOR (7 downto 0);
signal ADRI3 : STD_LOGIC_VECTOR (7 downto 0);
signal ADRI4 : STD_LOGIC_VECTOR (7 downto 0);
signal s_mux : STD_LOGIC_VECTOR (1 downto 0);
signal s_datos : STD_LOGIC_VECTOR (7 downto 0);
```

```
begin

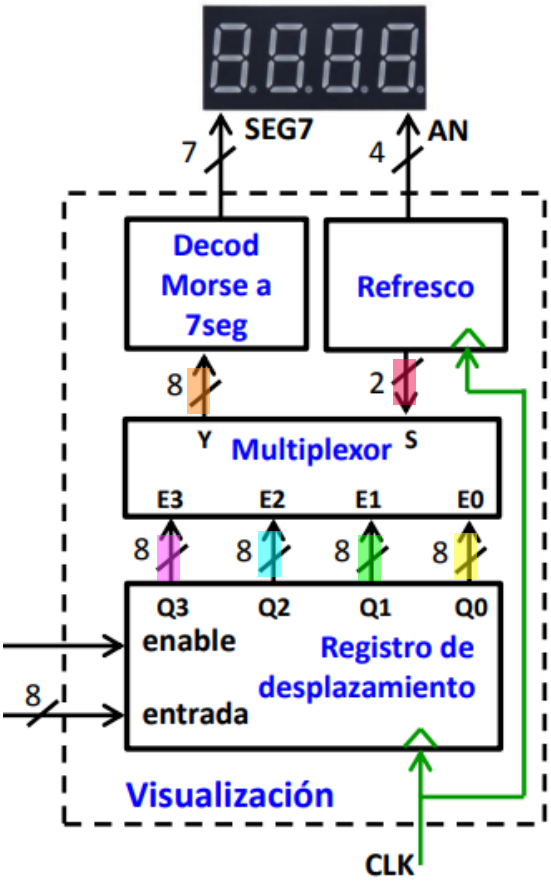
U1 : decodmorsea7s port map (     -- Las señales AN, SEG7, enable, entrada y CLK
se intercablean consigo mismas ya que entran o salen de la estructura general
del circuito completo
                CODIGO => s_datos,
                SEGMENTOS => SEG7
                );

   U2 : refresco port map (
                CLK => CLK,
                AN => AN,
                S => s_mux
                );

   U3 : MUX4x8 port map (
                E0 => ADRI1,
                E1 => ADRI2,
                E2 => ADRI3,
                E3 => ADRI4,
                S => s_mux,
                Y => s_datos
                );

   U4 : rdesp_disp port map (
                CLK=>CLK,
                ENTRADA=>ENTRADA,
                ENABLE=>ENABLE,
                Q0=>ADRI1,
                Q1=>ADRI2,
                Q2=>ADRI3,
                Q3=>ADRI4
                );

end a_visualizacion;
```



Detector de flanco

1. Detalle el código VHDL debidamente comentado.

```
entity detector_flanco is
    Port ( CLK      : in  STD_LOGIC;    -- reloj
           DAT      : in  STD_LOGIC;    -- Línea de datos
           LIN      : out  STD_LOGIC);  -- Valor detectado en el flanco
end detector_flanco;

architecture a_detector_flanco of detector_flanco is

constant UMBRAL0 : STD_LOGIC_VECTOR (7 downto 0):="00000101";  --5  umbral para
el 0
constant UMBRAL1 : STD_LOGIC_VECTOR (7 downto 0):="00001111";  --15 umbral para
el 1

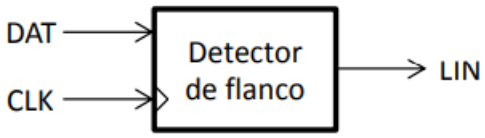
signal reg_desp : STD_LOGIC_VECTOR (19 downto 0):="00000000000000000000";
signal suma     : STD_LOGIC_VECTOR (7 downto 0) :="00000000";
signal s_valor  : STD_LOGIC:='0'; -- iniciamos las señales a 0

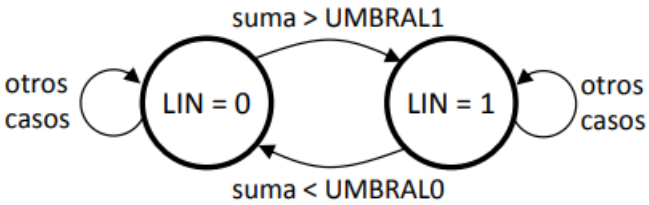
begin
  process (CLK)
    begin
      if (CLK'event and CLK='1') then -- a cada flanco de subida de reloj
desplazamos a la izquierda los bits del registro de desplazamiento
                suma <= suma + DAT - reg_desp(19); --calculamos el valor total
de todos los bits del registro de desplazamiento sumados, teniendo en cuenta el
valor del nuevo bit introducido y restando el del bit expulsado
                reg_desp(19) <= reg_desp(18);
                reg_desp(18) <= reg_desp(17);
                reg_desp(17) <= reg_desp(16);
                reg_desp(16) <= reg_desp(15);
                reg_desp(15) <= reg_desp(14);
                reg_desp(14) <= reg_desp(13);
                reg_desp(13) <= reg_desp(12);
                reg_desp(12) <= reg_desp(11);
                reg_desp(11) <= reg_desp(10);
                reg_desp(10) <= reg_desp(9);
                reg_desp(9) <= reg_desp(8);
                reg_desp(8) <= reg_desp(7);
                reg_desp(7) <= reg_desp(6);
                reg_desp(6) <= reg_desp(5);
                reg_desp(5) <= reg_desp(4);
                reg_desp(4) <= reg_desp(3);
                reg_desp(3) <= reg_desp(2);
                reg_desp(2) <= reg_desp(1);
                reg_desp(1) <= reg_desp(0);
                reg_desp(0) <= DAT;         --metemos el dato nuevo en el primer
bit
```

```
                if ( suma > UMBRAL1 ) then
                      s_valor <= '1' ;
                else if ( suma < UMBRAL0) then
                      s_valor <= '0' ;
                else
                      s_valor <= s_valor ;
                end if;
          end if;
       end if;

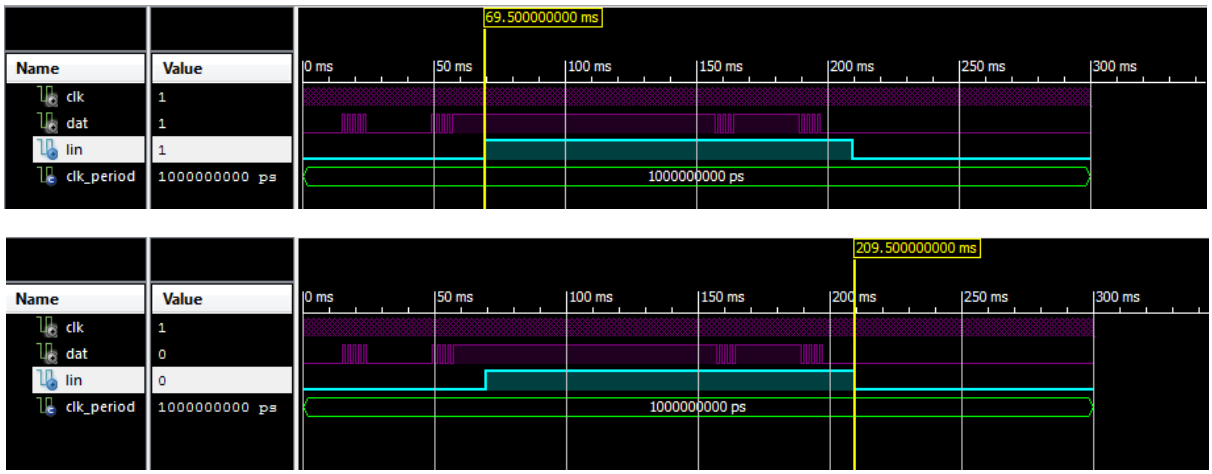
       end process;
       LIN<=s_valor;

end a_detector_flanco;
```





2. Incluya una captura de pantalla de la simulación donde puedan observarse los flancos de subida y bajada de la señal LIN y los tiempos en los que se producen.



Autómata de medición de duración.

1. Detalle el código VHDL debidamente comentado.

```
entity aut_duracion is
  Port ( CLK       : in  STD_LOGIC;     -- reloj de 1 ms
         LIN       : in  STD_LOGIC;     -- línea de entrada de datos
         VALID_INT : out  STD_LOGIC;    -- salida de validación de intervalo
         DATO      : out  STD_LOGIC;    -- salida de dato (0 o 1)
         DURACION  : out  STD_LOGIC_VECTOR (15 downto 0)); -- salida de
end aut_duracion;                                    -- duración del intervalo

architecture a_aut_duracion of aut_duracion is

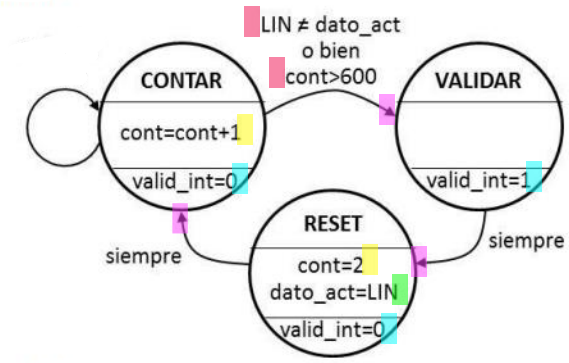
type STATE_TYPE is (CONTAR,VALIDAR,RESET);
signal ST : STATE_TYPE := RESET;
signal cont : STD_LOGIC_VECTOR (15 downto 0):="0000000000000000";
signal dato_act : STD_LOGIC:= '0';

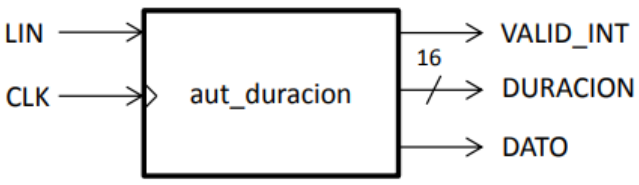
begin
process (CLK)  -- Autómata digital síncrono
  begin
    if (CLK'event and CLK='1') then
      case ST is
        when RESET =>
          cont<="0000000000000010";
          dato_act<=LIN;
          ST<=CONTAR;
            when CONTAR =>
                      cont<=cont+1;
                 if ((cont > 600) or (LIN /= dato_act)) then
                      ST<=VALIDAR;
                 end if;
            when VALIDAR =>
                 ST<=RESET;
      end case;
    end if;
  end process;

-- CABLEADOS DE LAS SALIDAS

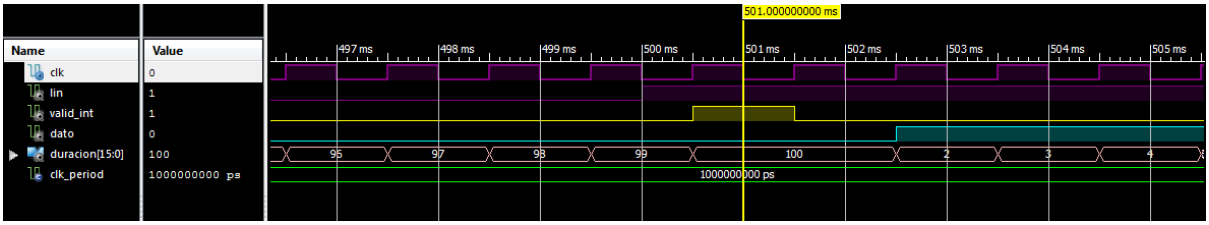
VALID_INT <='1' when ST=VALIDAR else '0';
DATO <= dato_act;
DURACION <= cont;

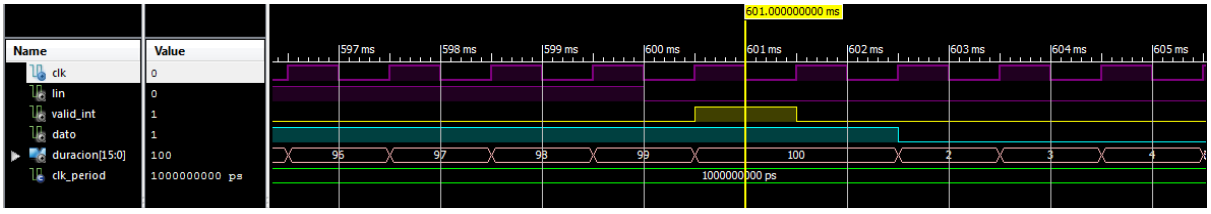
end a_aut_duracion;
```

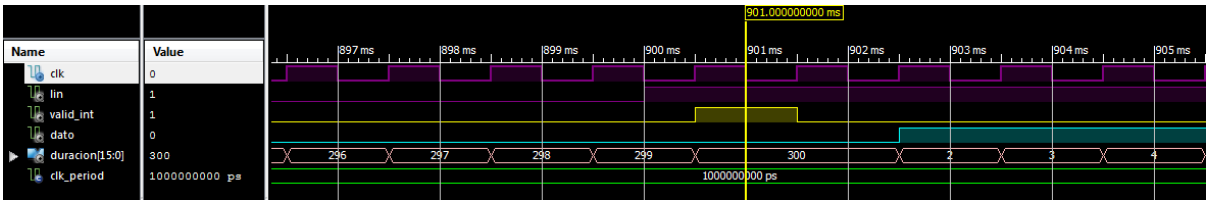


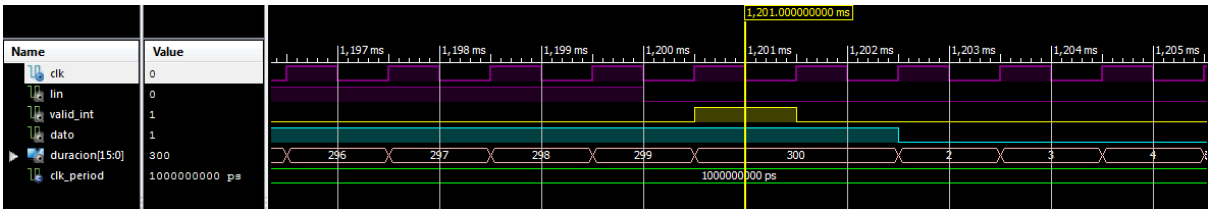


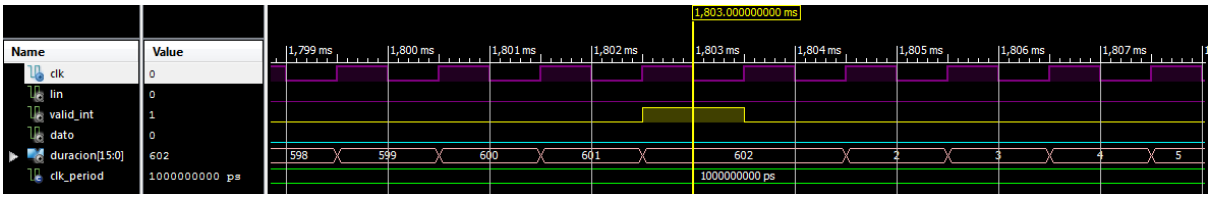
2. Incluya 7 capturas de pantalla donde se puedan apreciar con precisión los pulsos en la salida **VALID_INT** y los valores de las salidas **DATO** y **DURACION**. Emplee los cursores para marcar los tiempos.

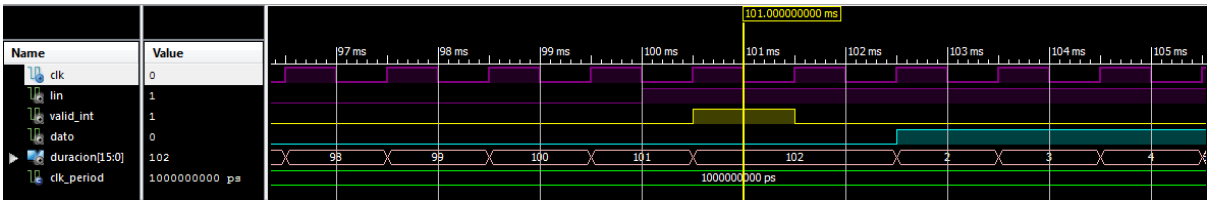


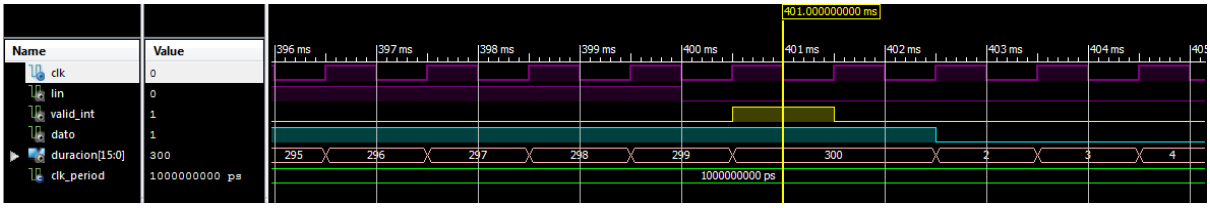












Estos son los 7 pulsos donde la señal valid_int vale 1

Autómata de control

1. Detalle el código VHDL debidamente comentado.

```
entity aut_control is
  Port ( CLK       : in STD_LOGIC;    -- reloj
       VALID_INT : in STD_LOGIC;    -- entrada de intervalo válido
       DATO      : in STD_LOGIC;    -- dato (0 o 1)
       DURACION  : in STD_LOGIC_VECTOR (15 downto 0); -- duración del intervalo
       CODIGO    : out  STD_LOGIC_VECTOR (7 downto 0); -- código morse obtenido
       VALID_COD : out  STD_LOGIC);              -- validación del código
end aut_control;

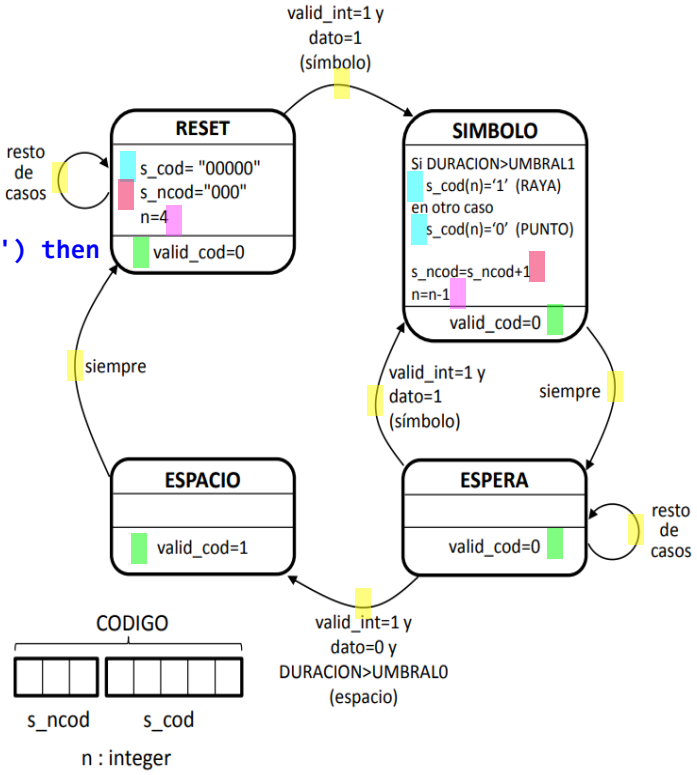
architecture a_aut_control  of aut_control is

constant UMBRAL0 : integer := 200;  -- umbral para los 0
constant UMBRAL1 : integer := 200;  -- umbral para los 1

type STATE_TYPE is (ESPACIO,RESET,SIMBOLO,ESPERA);

signal ST : STATE_TYPE := RESET;
signal s_ncod : STD_LOGIC_VECTOR (2 downto 0):="000"; --número de símbolos
que componen el código
signal s_cod  : STD_LOGIC_VECTOR (4 downto 0):="00000"; --código alineado a
la izquierda empleando "0" para punto y "1" para raya
signal n : INTEGER range 0 to 4; --posición de bit de s_cod a actualizar
begin

process (CLK)   -- Autómata digital síncrono
  begin
    if (CLK'event and CLK='1') then
      case ST is
        when RESET =>
          n <= 4;
          s_ncod<="000";
          s_cod<="00000";
          if (VALID_INT='1' and DATO='1') then
             ST<=SIMBOLO;
          else
             ST<=RESET;
          end if;
        when SIMBOLO =>
          n <= n-1;
          s_ncod <= s_ncod+1;
          if (DURACION>UMBRAL1) then
             s_cod(n)<='1';
          else
             s_cod(n)<='0';
          end if;
          ST<=ESPERA;
```



```
        when ESPERA =>
          if (VALID_INT='1' and DATO='1') then
            ST <= SIMBOLO;
          else if (VALID_INT='1' and DATO='0' and DURACION>UMBRAL0) then
            ST <= ESPACIO;
          end if;
          end if;
        when ESPACIO =>
          ST <= RESET;
      end case;
    end if;
  end process;

-- CABLEADOS DE LAS SALIDAS

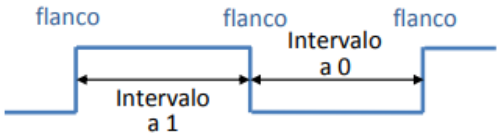
VALID_COD<='1' when ST=ESPACIO  else '0';
CODIGO(4 downto 0)<= s_cod ;              -- Complete las asignaciones
CODIGO(7 downto 5)<= s_ncod ;
end a_aut_control ;
```

2. Indique el valor elegido para las constantes **UMBRAL0** y **UMBRAL1** y la razón de su elección.

   Como dice el enunciado de la práctica : *"se definen dos umbrales (constantes) para los "0" y para los "1" que permiten distinguir si el intervalo a 0 se corresponde con una PAUSA o un ESPACIO (UMBRAL0) y si el intervalo a 1 se corresponde con un punto o una raya (UMBRAL1). Para ello, dichos umbrales se deben situar en el tiempo medio entre la duración corta y larga en ms."*

   Teniendo en cuenta :

   - un PUNTO (intervalo a 1 de 100 ms)
   - una RAYA (intervalo a 1 de 300 ms)
   - una PAUSA (intervalo a 0 de 100 ms)
   - un ESPACIO (intervalo a 0 de 300 ms)



   UMBRAL0 → $(t_{pausa}+t_{espacio})/2$ → (100+300)/2 = 200

   UMBRAL1 → $(t_{punto}+t_{raya})/2$ → (100+300)/2 = 200

3. Realice dos capturas de pantalla donde se puedan apreciar con precisión los pulsos en la salida **VALID_COD** y el valor de la salida **CODIGO**. Emplee los cursores para indicar los instantes temporales.



Circuito digital completo

1. Detalle el código VHDL debidamente comentado.

```
entity receptor is
    Port ( CLK_50MHz : in  STD_LOGIC;       -- Reloj de la FPGA (50 MHz)
           DAT  : in  STD_LOGIC;       -- Línea de entrada de datos
           SEG7 : out  STD_LOGIC_VECTOR (0 to 6);      -- Salida displays
           AN   : out  STD_LOGIC_VECTOR (3 downto 0)); -- Activación
end receptor;

architecture a_receptor of receptor is

component visualizacion is
  Port ( ENTRADA : in STD_LOGIC_VECTOR (7 downto 0);-- Entrada sig carácter
         ENABLE  : in STD_LOGIC;         -- Activación para desplazamiento
         CLK  : in STD_LOGIC;                        -- Entrada de reloj
         SEG7 : out STD_LOGIC_VECTOR (0 to 6);      -- Segmentos displays
         AN   : out STD_LOGIC_VECTOR (3 downto 0)); -- Activación displays
end component;




component aut_control is
  Port ( CLK       : in STD_LOGIC;    -- reloj
                   VALID_INT : in STD_LOGIC;-- entrada de intervalo v?lido
                   DATO      : in STD_LOGIC;    -- dato (0 o 1)
```

```
                    DURACION   : in STD_LOGIC_VECTOR (15 downto 0);         -
- duración del intervalo
                    CODIGO     : out  STD_LOGIC_VECTOR (7 downto 0); -- codigo
morse obtenido
                    VALID_COD : out  STD_LOGIC);
end component;

component aut_duracion is
  Port ( CLK       : in  STD_LOGIC;     -- reloj de 1 ms
         LIN       : in  STD_LOGIC;     -- línea de entrada de datos
         VALID_INT : out  STD_LOGIC;    -- salida de validación de intervalo
         DATO      : out  STD_LOGIC;    -- salida de dato (0 o 1)
         DURACION  : out  STD_LOGIC_VECTOR (15 downto 0));
end component;

component detector_flanco is
  Port ( CLK      : in  STD_LOGIC;     -- reloj
         DAT      : in  STD_LOGIC;     -- Línea de datos
         LIN      : out  STD_LOGIC);  -- Valor detectado en el flanco
end component;

component div_reloj is
    Port ( CLK_50MHz : in  STD_LOGIC;       -- Entrada reloj 50 MHz, T=2ns
           CLK       : out  STD_LOGIC);     -- Salida reloj 1 KHz, T=1ms
end component;

signal ANDRES1 : STD_LOGIC;    --Señales de interconexion entre modulos
signal ANDRES2 : STD_LOGIC;
signal ANDRES3 : STD_LOGIC;
signal ANDRES4 : STD_LOGIC_VECTOR (15 downto 0);
signal ANDRES5 : STD_LOGIC_VECTOR (7 downto 0);
signal ANDRES6 : STD_LOGIC;
signal reloj : STD_LOGIC;

begin

  U1 : visualizacion port map (
                    CLK => reloj,
                    ENTRADA => ANDRES5,
                    ENABLE => ANDRES6,
                    SEG7 => SEG7,
                    AN => AN
                    );

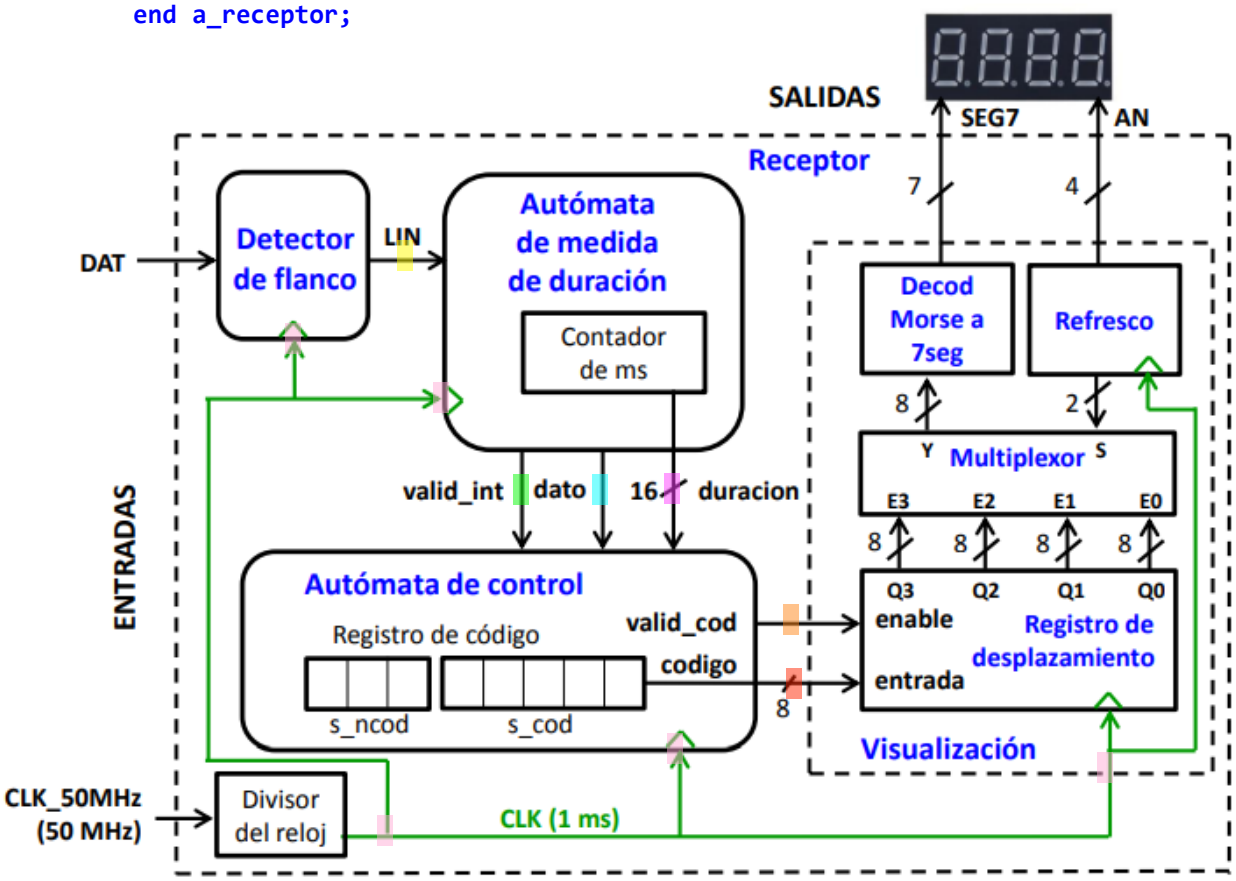

      U2 : aut_control port map (
                    CLK => reloj,
                    VALID_INT => ANDRES2,
                    DATO => ANDRES3,
                    DURACION => ANDRES4,
                    CODIGO => ANDRES5,
```

```
                    VALID_COD => ANDRES6
                    );

        U3 : aut_duracion port map (
                    CLK => reloj,
                    LIN => ANDRES1,
                    VALID_INT => ANDRES2,
                    DATO => ANDRES3,
                    DURACION => ANDRES4
                    );

        U4 : detector_flanco port map (
                    CLK => reloj,
                    LIN => ANDRES1,
                    DAT => DAT
                    );

        U5 : div_reloj port map (
                    CLK_50MHz => CLK_50MHz,
                    CLK => reloj
                    );
end a_receptor;
```



Comentarios y puntualizaciones que desee añadir